

Agent Skills für KI-Coding-Agenten: Funktionsweise, Evidenzlage, Aufbau und ein evidenzbasierter Leitfaden

Lucas Tuscher
Student
Stand: 4. Oktober 2026

Zusammenfassung—KI-Coding-Agenten benötigen in großen Codebasen projektspezifisches Wissen. *Agent Skills* bündeln Anweisungen, Referenzen und Skripte, die ein Agent bei Bedarf lädt; seit Dezember 2025 ist das Format ein offener Standard. Diese Literaturübersicht ordnet 33 Quellen zu Funktionsweise, Wirksamkeit, Kontextkosten, Rückkopplung und Sicherheit ein. In der aktuellen SkillsBench-Fassung erhöhen kuratierte Skills die Erfolgsrate im Mittel um 16,6 Prozentpunkte, in der Softwareentwicklung um 11,6; einzelne Aufgaben verschlechtern sich. Die Aktivierung hängt vom Werkzeug und Versuchsaufbau ab. Kontextdateien verbessern den Erfolg nicht generell und können zusätzliche Kosten verursachen. Eine Sicherheitsstudie klassifiziert 26,1 % der untersuchten öffentlichen Skills als potenziell verwundbar. Daraus wird ein Leitfaden mit gekennzeichneteter Evidenzstärke abgeleitet, ergänzt um Test- und Freigabeverfahren sowie einen illustrativen Beispiel-Skill.

Schlagwörter—Agent Skills, KI-Coding-Agenten, Context Engineering, Software-Qualität, Large Language Models

I. EINLEITUNG

Der Nutzen KI-gestützter Softwareentwicklung ist empirisch widersprüchlich. Kontrollierte Experimente berichten von 55,8 % schnellerer Bearbeitung einer abgegrenzten Aufgabe [1], rund 21 % geschätzter Zeitersparnis bei Google mit breitem Konfidenzintervall [2] und 26 % mehr abgeschlossenen Aufgaben in drei Feldexperimenten mit fast 5.000 Entwickler:innen [3]. Demgegenüber benötigten erfahrene Open-Source-Entwickler:innen in einer randomisierten Studie mit KI-Werkzeugen 19 % *mehr* Zeit für Aufgaben in ihren eigenen, ausgereiften Projekten – obwohl sie selbst eine Beschleunigung von 20 % wahrnahmen [4]. DORA beschreibt KI als *Verstärker* organisatorischer Stärken und Schwächen; höhere KI-Nutzung ist im Bericht auch mit größerer Auslieferungsinstabilität verbunden [5]. GitClear beobachtet im selben Zeitraum mehr duplizierten Code und weniger Refactoring [6]. Diese Beobachtungen belegen für sich keine kausale Wirkung von KI.

Eine mögliche Erklärung für einen Teil dieser Unterschiede ist der Kontext: Generische Modelle kennen projektspezifische Regeln und Fallstricke nicht zuverlässig. Die Produktivitätsstudien isolieren diese Ursache jedoch nicht. *Agent Skills* setzen am fehlenden prozeduralen Wissen an und laden es aufgabenbezogen nach [7].

Diese Arbeit ist eine Literatursynthese mit Leitfaden. Sie führt keine eigenen Messungen durch, sondern stützt sich ausschließlich auf veröffentlichte Studien und Dokumentatio-

nen und kennzeichnet bei jeder Aussage, wie belastbar der Beleg ist. Sie beantwortet vier Fragen:

- **RQ1:** Wie funktionieren Agent Skills technisch, und wie grenzen sie sich von anderen Mechanismen ab?
- **RQ2:** Welche Vorteile und Grenzen belegt die empirische Evidenz?
- **RQ3:** Wie sind Skills typischerweise aufgebaut, und was folgt aus der Evidenz für ihren Entwurf?
- **RQ4:** Wie setzt man Skills im Unternehmen verlässlich, kosteneffizient und sicher ein?

Tabelle I zeigt Lesepfade für unterschiedliche Leserschaften.

Tabelle I
Lesepfade durch die Arbeit

Wenn du ...	dann lies ...
Skills zum ersten Mal kennenlernen	Abschnitt III, danach Abschnitt IV.B
einen Skill bauen willst	Abschnitt VI und Abschnitt IX
Skills im Unternehmen einführen sollst	Abschnitt V, Abschnitt VI.I, Abschnitt VII
die Belege prüfen willst	Abschnitt II und Abschnitt IV

II. METHODIK UND EVIDENZBASIS

Die Arbeit ist eine narrative Literaturübersicht mit Quellenprüfung zum 4. Oktober 2026. Die Auswahl umfasst arXiv-Arbeiten, Konferenz- und Zeitschriftenbeiträge, Herstellerdokumentationen und technische Berichte. Ein vollständiges Such- und Screeningprotokoll liegt nicht vor; die Übersicht beansprucht daher weder systematische Vollständigkeit noch den methodischen Status eines reproduzierbaren Rapid Review.

Suchbegriffe: „agent skills“, „SKILL.md“, „AGENTS.md“, „context engineering“, „coding agent“, „tool selection“, „skill security“, „long context degradation“.

Einschlusskriterien: Veröffentlichung ab 2022; messbare Aussage zu Skills, Kontextdateien, Werkzeugauswahl, Kontextlänge, Rückkopplungsschleifen, Produktivität oder Sicherheit von Coding-Agenten, oder Primärdokumentation des Skill-Formats.

Tabelle II
Evidenzklassen der 33 Quellen; zitierte Fassungen laut Literaturverzeichnis

Evidenzklasse	Zahl	Quellen
Begutachtete Konferenz-/ Zeitschriftenbeiträge	7	[2], [8], [9], [10], [11], [12], [13]
Fachzeitschriftenbeitrag	1	[14]
Workshop-Beitrag	1	[15]
arXiv-Preprints	11	[1], [4], [16], [17], [18], [19], [20], [21], [22], [23], [24]
Working Paper	1	[3]
Branchen- und technische Berichte	3	[5], [6], [25]
Standard-, Hersteller- und Projektdokumentation	9	[7], [26], [27], [28], [29], [30], [31], [32], [33]

Tabelle III
Die drei Ladestufen der progressiven Offenlegung

Stufe	Inhalt	Wann geladen	Größe
1	Name und Beschreibung	bei der Skill-Erkennung; Sichtbarkeit werkzeugabhängig	ca. 100 Token je Skill
2	Rumpf der SKILL.md	wenn die Aufgabe passt	empfohlen unter 5.000 Token bzw. 500 Zeilen
3	Referenzen und Skripte	bei Bedarf; reine Ausführung benötigt nur die Ausgabe im Kontext	kein Formatlimit; Kontext- und Ausführungsgrenzen bleiben

Ausschlusskriterien: reine Meinungsbeiträge ohne Messung und Arbeiten ohne Bezug zu Code-Agenten.

Die Auswahl umfasst 33 Quellen. Tabelle II ordnet sie nach Evidenzklasse. Die vier unmittelbar skillspezifischen Studien werden als Preprints zitiert. Aussagen über Skills sind daher weniger gesichert als Aussagen über Kontextlänge oder Rückkopplung, für die begutachtete Arbeiten vorliegen.

Zwei Verzerrungen sind zu beachten. Erstens stammen neun Quellen aus Standard-, Hersteller- oder Projektdokumentation; konkrete Werkzeugdetails sind deshalb nicht unabhängig belegt. Zweitens gelten empirische Zahlen jeweils für die untersuchten Modelle, Werkzeuge und Aufgaben. Bei SkillsBench wird die aktuelle Fassung v4 verwendet; die überarbeitete arXiv-Fassung v3 von Gloaguen et al. wird ergänzend zum Workshop-Beitrag berücksichtigt.

Zur Stärke einzelner Aussagen verwendet der Leitfaden (Abschnitt VI) drei Stufen: **stark** (mehrere unabhängige Studien, mindestens eine begutachtet), **mittel** (eine empirische Studie oder mehrere Preprints) und **schwach** (Herstellerempfehlung oder Einzelmessung eines Herstellers).

III. GRUNDLAGEN

A. Aufbau eines Skills

Ein Skill ist ein Verzeichnis mit einer SKILL.md. Der YAML-Kopf enthält im offenen Standard `name` (1–64 Zeichen, Kleinbuchstaben, Ziffern und Bindestriche; weder Rand- noch Doppelbindestriche; identisch mit dem Verzeichnisnamen) und `description` (1–1.024 Zeichen) [26]. Optional kommen `scripts/`, `references/` und `assets/` hinzu. Anthropic stellte Skills im Oktober 2025 vor und veröffentlichte das Format im

Dezember 2025 als offenen Standard [7]. Installationspfade und zusätzliche Felder hängen vom Werkzeug ab; Claude Code verwendet beispielsweise `.claude/skills/` [28]. Ein einheitlicher Installationspfad wird durch das Format nicht vorgeschrieben.

B. Progressive Offenlegung

Skills werden in drei Stufen geladen [7], [26] (Tabelle III): Damit kann ein Skill umfangreiche Ressourcen bündeln, ohne sie vollständig in jede Anfrage zu laden. Name und Beschreibung helfen bei der Auswahl; manuelle Aufrufe, Sichtbarkeitsregeln oder zusätzliche Hinweise können die Aktivierung ebenfalls beeinflussen.

C. Ablauf zur Laufzeit

Abb. 1 zeigt einen typischen modellgesteuerten Ablauf [7]. Der Agent wählt anhand der verfügbaren Beschreibungen, ob er einen Skill liest oder aufruft. Das Format schreibt weder einen bestimmten Router noch einen Stichwort-Abgleich vor. Explizite Nutzeraufrufe und werkzeugspezifische Filter sind ebenfalls möglich [28].

Tabelle IV
Mechanismen zur Steuerung von Coding-Agenten. Zeilen ohne Quelle sind einordnende Beschreibungen des Autors.

Mechanismus	Ladeverhalten	Geeignet für	Schwäche
CLAUDE.md / AGENTS.md, Cursor Rules, Copilot Instructions	je nach Werkzeug, Verzeichnis und Regeltyp	wenige Regeln, die jede Aufgabe betreffen	kostet bei jeder Anfrage; kein genereller Erfolgsgewinn [15]
Skill	bei passender Aufgabe, vom Modell gewählt	Abläufe, Fachwissen, Prüfskripte	Aktivierung unzuverlässig [17], [31]
Slash-Command / Prompt-Vorlage	nur auf expliziten Aufruf durch den Menschen	bewusst gestartete, wiederkehrende Aufgaben	kein automatisches Laden
Hook	bei Ereignis, deterministisch	Pflichtprüfungen, die nie vergessen werden dürfen [29]	starr, kein Urteilsvermögen
MCP-Werkzeug	Beschreibungen direkt oder per Werkzeug-Erkennung; Aufruf bei Bedarf	Zugriff auf externe Systeme	viele Werkzeugbeschreibungen belasten die Auswahl [21]
Subagent	eigener Kontext	breite Suche, Isolation langer Ausgaben	hoher Tokenverbrauch [30]
RAG / Dokumentationssuche	Abruf per Ähnlichkeitssuche	große, wechselnde Wissensbestände	Abrufqualität und Aktualität; Abläufe müssen zusätzlich gestaltet werden
Fine-Tuning	durch Training in Modellgewichten verankert	dauerhaft stabiles Verhalten und Stil	teuer, nicht pro Repository änderbar

1. Start

Das Werkzeug stellt die sichtbaren Skill-Namen und Beschreibungen bereit (Stufe 1).



2. Anfrage

Der Nutzer stellt eine Aufgabe. Das Modell vergleicht sie mit den Beschreibungen.



3. Entscheidung

Passt ein Skill, ruft das Modell ihn auf oder liest die SKILL.md per Dateizugriff (Stufe 2). Passt keiner, bleibt der Skill ungenutzt.



4. Ausführung

Das Modell folgt dem Ablauf und liest Ressourcen bei Bedarf (Stufe 3). Bei reiner Skriptausführung muss nur die Ausgabe in den Kontext gelangen.



5. Rückkopplung

Prüfskripte melden Funde. Das Modell korrigiert und prüft erneut; bei ungelösten Problemen oder erreichtem Rundenlimit berichtet es den offenen Stand.

D. Geltungsbereiche, Vorrang und Konflikte

Skills können im Repository, im Benutzerverzeichnis oder über Organisationsverwaltung bereitgestellt werden. Gleichnamige Skills unterliegen werkzeugspezifischen Vorrangregeln [28]. Für inhaltliche Konflikte zwischen *verschiedenen* Skills belegen die herangezogenen Quellen keinen einheitlichen Mechanismus. Als Entwurfsheuristik empfiehlt sich, Überschneidungen zu vermeiden und die maßgeblichen Projektregeln ausdrücklich zu benennen. Ein Satz im Skill ersetzt dabei nicht die tatsächliche Instruktionshierarchie des Werkzeugs.

E. Abgrenzung zu verwandten Mechanismen

Tabelle IV ordnet Skills in das Umfeld der Agentenkonfiguration ein. Entscheidend ist der Unterschied zwischen *immer geladenem* Kontext, *bei Bedarf geladenem* Wissen und *deterministisch ausgeführten* Prüfungen.

Skills und RAG sind keine Gegensätze: Ein Skill kann auf eine Suchfunktion oder ein Werkzeug verweisen. RAG beantwortet „Was steht in der Dokumentation?“, ein Skill beschreibt „Wie gehen wir vor, und woran messen wir Qualität?“.

F. Typischer inhaltlicher Aufbau

Die Spezifikation [26] und die Herstellerempfehlungen [27] ergeben das Grundmuster in Tabelle V. Die Tabelle beschreibt Empfehlungen, keine Häufigkeitsverteilung. Skills-Bench v4 berichtet ergänzend Größenstatistiken: Der Median der SKILL.md liegt in der erreichbaren Stichprobe bei 4,8 KB, der des gesamten Pakets bei 7,2 KB [16]. Diese Stichprobe ist keine repräsentative Vollerhebung aller Skills.

Abb. 1. Ablauf der Skill-Nutzung zur Laufzeit

Tabelle VI
Vorteile und Grenzen von Agent Skills

Vorteile	Grenzen
Wirksam, wenn kuratiert: +16,6 Prozentpunkte im Mittel [16]	Selbstgenerierte Skills in drei Konfigurationen schlechter als die Basislinie; 13 von 87 Aufgaben werden schlechter; in der Softwareentwicklung nur +11,6 Prozentpunkte [16]
Kontextsparend durch progressive Offenlegung (Tabelle III)	Aktivierung unzuverlässig: in einer Herstellermessung 56 % nie aufgerufen [31]
Wirken als wiederholbarer Ablauf, nicht als Lexikon [17]	Auswahl verschlechtert sich mit der Zahl der Skills [17]
Kleinere Modelle mit Skill können stärkere Basislinien ohne Skill erreichen [16]	Modelle folgen Skill-Anweisungen unterschiedlich treu [18]
Deterministische Prüfung über Skripte; Rückkopplung wirkt stark [22], [23]	Skripte sind Code: Fehlalarme und Sicherheitsrisiken (Abschnitt V); Skills mit Skripten häufiger verwundbar [19]
Wiederverwendbar, im Repository versionierbar, im Team teilbar	Pflegeaufwand; veraltete Skills können unbemerkt schaden (Schlussfolgerung des Autors, nicht gemessen)
Offener, werkzeugübergreifender Standard [26]	Werkzeugspezifische Zusatzfelder und Limits [28]
Kein Training nötig; sofort änderbar	zusätzlicher Token- und Zeitverbrauch möglich (Abschnitt IV.F)

Tabelle V
Empfohlene Bausteine einer SKILL.md

Baustein	Inhalt
YAML-Kopf	name, description (Pflicht); optional Lizenz, Voraussetzungen, Metadaten
Ziel	ein Satz, was der Skill erreichen soll
Ablauf	nummerierte Schritte, die der Agent abarbeitet
Regeln	kurze Aufzählung, idealerweise mit Begründung
Gotchas	bekannte Fehler mit Korrektur
Ausgabeformat	Inhalt und Länge des Berichts
Verweise	Referenzen mit Hinweis, wann sie zu lesen sind; Skripte mit Hinweis, ob ausführen oder lesen

IV. ERGEBNISSE DER LITERATURANALYSE

A. Wirksamkeit von Skills

Die aktuelle Fassung von *SkillsBench* (v4) umfasst 87 Aufgaben aus acht Domänen und 18 Modell-Werkzeug-Konfigurationen [16]. Im Vergleich ohne bzw. mit kuratierten Skills steigt die mittlere Erfolgsrate von 33,9 % auf 50,5 % (+16,6 Prozentpunkte). In der Softwareentwicklung beträgt der Zugewinn 11,6 Prozentpunkte; 13 von 87 Aufgaben verschlechtern sich. Selbstgenerierte Skills werden separat in drei Konfigurationen untersucht und schneiden dort schlechter ab als die jeweilige Basislinie. Das ist kein Beleg, dass Modelle grundsätzlich keine nützlichen Skills entwerfen können: Die Autoren nennen auch Probleme bei Erkennung und Trennung von Erzeugungs- und Lösungsphase. Kompakte Skills und wenige passende Skills pro Aufgabe schneiden besser ab als umfassende Pakete; kleinere Modelle mit Skills können stärkere Basislinien ohne Skills erreichen.

Shaposhnikov et al. werten 500 reale Skills mit 1.000 abgeleiteten Aufgaben über 19 Agent-Modell-Konfigurationen

aus und zeigen, dass Modelle sich stark darin unterscheiden, wie treu sie Skill-Anweisungen folgen [18]. Jiang et al. normalisieren 8.135 Versuchsdatensätze und klassifizieren 238 gültige Labels aus 240 qualitativ codierten Datensätzen: Prozedurale Anker machen 65,7 % der Skill-Fälle aus, explizite Wissensinjektion 4,5 % [17]. Diese Anteile beschreiben Nutzungsweisen, keine Erfolgsraten aller Durchläufe. Alle drei Studien werden als Preprints zitiert.

B. Vorteile und Grenzen im Überblick

Tabelle VI fasst zusammen, was für und gegen Skills spricht, jeweils mit Beleg.

C. Aktivierung: das unterschätzte Problem

Ein Skill kann über seine Inhalte wirken, wenn diese tatsächlich zugänglich werden. Vercel meldet in einem Next.js-16-Setup 56 % nicht aufgerufene Skills; ein 8-KB-Dokumentationsindex in *AGENTS.md* erreichte 100 % Erfolg, ein explizit angeforderter Skill 79 % [31]. *SkillsBench* v4 zeigt dagegen, dass Erkennung in vielen getesteten Konfigurationen nicht der Hauptengpass ist [16]. Aktivierung ist somit abhängig vom Setup. Jiang et al. berichten bei fünf bis 100 angebotenen Skills einen Rückgang der Präzision der tatsächlichen Nutzung von 29,6 % auf 3,3 %, ohne entsprechenden allgemeinen Einbruch des Aufgabenerfolgs [17]. Diese Präzision ist von Aktivierungsrate und Erfolgsrate zu unterscheiden. Bei MCP-Werkzeugen verbessert Retrieval die Auswahlgenauigkeit von 13,62 % auf 43,13 % und reduziert Prompt-Token um mehr als 50 % [21].

Daraus folgt als Entwurfsheuristik: Beschreibungen sollten konkrete Aufgaben und Grenzen nennen. Bei Aktivierungsproblemen kann ein kurzer Verweis in dauerhaft verfügbarem Projektkontext helfen. Messwerte verschiedener Quellen sind wegen unterschiedlicher Werkzeuge, Aufgaben und Definitionen nicht unmittelbar vergleichbar (Abschnitt VII).

D. Kontextbudget und Regeldateien

Mehr Kontext ist nicht automatisch besser. Gloaguen et al. untersuchen SWE-bench-Aufgaben sowie 138 CTX-bench-Aufgaben; die aktualisierte Fassung v3 findet keinen generellen Erfolgsgewinn durch Kontextdateien, aber durchschnittlich über 20 % höhere Inferenzkosten [15]. Daraus folgt kein pauschales Verbot von AGENTS.md: Nichtstandardisierte Projektregeln können weiterhin nützlich sein. IFScale misst bei 500 Keyword-Anweisungen für Geschäftsberichte selbst bei den besten Modellen nur 68 % Befolgung und eine Bevorzugung früher Anweisungen [20]. Diese künstliche Aufgabe ist kein direkter Coding-Benchmark. Context Rot [25] und Lost in the Middle [8] zeigen ebenfalls Kontextprobleme in ihren jeweiligen Aufgaben. Gemeinsam motivieren die Befunde knappe, relevante Anweisungen; sie beweisen keine universelle optimale Skill-Länge.

E. Rückkopplungsschleifen schlagen Anweisungen

Anweisungen allein garantieren keine Qualität; Prüfwerkzeuge mit Rückmeldung an das Modell wirken stärker. Blyth et al. lassen GPT-4o Code iterativ anhand statischer Analyse (Bandit, Pylint) verbessern: Sicherheitsprobleme sinken von über 40 % auf 13 %, Lesbarkeitsverstöße von über 80 % auf 11 % und Zuverlässigkeitswarnungen von über 50 % auf 11 % innerhalb von zehn Iterationen [22]. Für C++ zeigen Tran et al. an 3,52 Millionen produktiven Code-Änderungen, dass KI-generierter Code überdurchschnittlich viele Kopier- und Allokationskosten, Kopplungsprobleme und handgeschriebene Schleifen statt Standardalgorithmen enthält, was 5–8 % mehr Rechenressourcen kostet; gezieltes, taxonomiebasiertes Feedback senkte die betroffenen Warnungen um 11,1 % [23]. Allgemeiner belegen *Self-Refine* [9] und *Reflexion* [10], dass Modelle mit Rückmeldung über mehrere Runden deutlich bessere Ergebnisse erzielen. Wie stark die Gestaltung der Schnittstelle zwischen Agent und Umgebung zählt, zeigt SWE-agent: Speziell entworfene Befehle mit kompakter Ausgabe verbessern die Lösungsrate realer GitHub-Issues erheblich [11]. Die Arbeiten untersuchen Rückkopplung allgemein, nicht speziell in Skills; dass Skripte in Skills denselben Effekt haben, ist eine plausible Übertragung, aber nicht direkt gemessen.

Ein Skill sollte daher nicht nur beschreiben, wie guter Code aussieht, sondern Skripte mitliefern, die Abweichungen messen, und eine Schleife vorschreiben: prüfen, korrigieren, erneut prüfen.

F. Token-Ökonomie und Subagenten

Anthropic berichtet für sein Recherchesystem 90,2 % höhere Leistung als bei einem Einzelagenten und ungefähr den 15-fachen Tokenverbrauch einer normalen Unterhaltung; Tokenmenge erklärt im dortigen Vergleich 80 % der Leistungsvarianz [30]. Diese Herstellerzahlen sind keine Coding-Messung und keine kausale Kostenformel. Gezielte Suche, kompakte Werkzeugausgaben und kurze Berichte können den aktiven Kontext begrenzen; tatsächliche Kosten hängen auch von Modell, Caching und Kontextverwaltung ab. SkillsBench v4 enthält Token- und Kostenauswertungen, deren Werte an die untersuchten Konfigurationen gebunden sind [16].

V. SICHERHEIT

Skills können das Verhalten und die Codeausführung des Agenten beeinflussen. Liu et al. sammeln 42.447 öffentliche Skills und analysieren davon 31.132: Ihr Detektor klassifiziert 26,1 % als mindestens einmal verwundbar und 5,2 % als mit hochkritischen, möglicherweise absichtlichen Mustern versehen [19]. Für Skills mit Skripten beträgt das Odds Ratio 2,12; das ist kein Verhältnis der Verwundbarkeitswahrscheinlichkeiten. Die Detektorwerte (86,7 % Präzision, 82,5 % Recall) begrenzen die Aussagekraft; die Zahlen sind keine bestätigte Quote aller öffentlich verfügbaren Skills. Daneben fanden Pearce et al. rund 40 % verwundbare Copilot-Programme in ausgewählten sicherheitsrelevanten Szenarien [12]. Kozak et al. untersuchen unsichere Agentenaktionen bei Software-Setup-Aufgaben [24].

A. Bedrohungsmodell

Ein Skill kann auf vier Wegen schaden: (1) *Prompt Injection* in SKILL.md oder Referenzen, die den Agenten zu fremden Zielen lenkt; (2) *Datenabfluss* über Skripte oder Netzwerkzugriffe; (3) *Rechteauserweiterung*, wenn ein Skill mehr Werkzeuge nutzt, als seine Aufgabe braucht; (4) *Lieferkettenangriffe* über nachgeladene Abhängigkeiten oder unbemerkte Änderungen nach der Freigabe. Diese vier Klassen folgen der Einteilung bei [19].

B. Gegenmaßnahmen

Tabelle VII ordnet jeder Bedrohung eine Gegenmaßnahme zu. Die Zuordnung ist eine Ableitung des Autors aus allgemeinen Sicherheitsprinzipien; die Wirksamkeit dieser Maßnahmen für Skills wurde in den verwendeten Quellen nicht gemessen.

VI. LEITFADEN: SKILLS RICHTIG BAUEN UND EINSETZEN

Dieser Abschnitt übersetzt die Befunde in Regeln, die direkt als interne Richtlinie übernommen werden können. Tabelle VIII zeigt zuerst, wie gut jede Kernregel belegt ist, damit Teams wissen, welche Regeln sie unbedingt einhalten und welche sie an ihren Kontext anpassen können.

A. Wohin gehört welches Wissen?

Der häufigste Entwurfsfehler ist nicht ein schlechter Skill, sondern Wissen am falschen Ort. Tabelle IX ordnet typische Inhalte dem passenden Mechanismus zu.

Tabelle VII
Bedrohungen und Gegenmaßnahmen für Skills

Bedrohung	Gegenmaßnahme
Prompt Injection	Review aller Dateien des Skills, auch der Referenzen; keine Skills aus ungeprüften Quellen; Anweisungen zum Ignorieren von Regeln oder Verstecken von Ausgaben als Warnsignal
Datenabfluss	Skripte ohne Netzwerkzugriff; Ausführung in einer Sandbox; keine Zugangsdaten im Skill, stattdessen Secret-Store
Rechteauserweiterung	Prinzip der minimalen Rechte; Berechtigungen im ausführenden Werkzeug begrenzen; <code>allowed-tools</code> ist keine Sandbox; schreibende Aktionen nur mit ausdrücklicher Option
Lieferkette	Abhängigkeiten festpinnen; Skills nur aus interner Registry mit Versionierung und Code-Review; Änderungen per Pull Request
Unbemerkte Änderung	Prüfsummen oder Signierung der Skill-Verzeichnisse; Änderungen im Audit-Log

Tabelle VIII
Evidenzstärke der Kernregeln des Leitfadens

Regel	Beleg	Stärke
Kurz halten, Wichtigstes nach vorn, nur Projektwissen	Kein genereller Gewinn durch Kontextdateien [15]; Befolgung sinkt mit Zahl der Anweisungen [20]; Leistung sinkt mit Länge [8], [25]	stark
Prüfskripte mit Schleife „prüfen – korrigieren – erneut prüfen“	Rückkopplung wirkt [9], [10], [11], [22], [23] (allgemein, nicht skillspezifisch)	stark
Modellentwürfe kuratieren und gegen eine Basislinie prüfen	[16]	mittel
Wenige passende Skills pro Aufgabe; kompakte Inhalte	[16]	mittel
Als Ablauf schreiben, nicht als Lexikon	[17] (65,7 % prozedurale Anker)	mittel
Wenige, klar abgegrenzte Skills	Präzision der Nutzung sinkt bei großen Skill-Pools [17], [21]	mittel
Skills vor Freigabe auf Sicherheit prüfen	26,1 % vom Detektor als verwundbar klassifiziert [19]; Risiken generierten Codes [12], [24]	mittel
Beschreibung mit Auslösern und Abgrenzung	Aktivierung kann ein Engpass sein [17], [31]; die Formel selbst ist Herstellerempfehlung [27]	mittel bis schwach
Pflichtwissen zusätzlich im Immer-Kontext oder Hook	Einzelmessung eines Herstellers [31]	schwach
Freiheitsgrad anpassen; Referenzen eine Ebene tief; Inhaltsverzeichnis ab 100 Zeilen	Herstellerempfehlung [27], nicht unabhängig geprüft	schwach

Tabelle IX
Entscheidungshilfe für den Ablageort von Wissen

Das Wissen ...	gehört in
gilt für jede Aufgabe und ist in wenigen Zeilen sagbar (Build-Befehl, Schichtregel)	CLAUDE.md / AGENTS.md
ist ein Ablauf oder Fachwissen für eine Aufgabenfamilie	Skill
darf nie vergessen werden (Formatierung, Pflichtprüfung)	Hook oder CI
braucht Zugriff auf ein externes System (Ticket, Datenbank)	MCP-Werkzeug
ist ein großer, wechselnder Faktenbestand (API-Doku, Wiki)	Suche/RAG, im Skill nur verlinkt
erfordert breite Suche mit viel Zwischenausgabe	Subagent
betrifft nur die aktuelle Aufgabe	Prompt

Ein Skill ersetzt also keine Regeldatei und keinen Hook; er ergänzt sie. Bei wiederkehrenden Aktivierungsproblemen kann ein kurzer Verweis im dauerhaften Kontext helfen [31]. Verbindliche Prüfungen gehören in unabhängig konfigurierte Hooks oder CI; ein erst mit dem Skill geladener Hook hilft bei ausbleibender Aktivierung nicht.

B. Verzeichnisstruktur

Als Entwurfsheuristik deckt ein Skill *eine* Aufgabenfamilie ab, etwa „Datenbankmigrationen“, nicht „alles zum Backend“. Listing 1 zeigt eine bewährte Struktur.

Tabelle X
Felder im Kopf einer SKILL.md

Feld	Regel
name	1–64 Zeichen, nur Kleinbuchstaben, Ziffern und Bindestriche; keine Rand- oder Doppelbindestrich; gleich dem Ordernamen. Reservierte Namen sind werkzeugspezifisch.
description	1–1.024 Zeichen; beschreibt <i>was</i> und <i>wann</i> . Dritte Person ist eine Herstellerempfehlung, keine Formatpflicht.
license	Lizenz oder Vertraulichkeitshinweis
compatibility	1–500 Zeichen; besondere Voraussetzungen wie Python-Version oder Netzwerk
metadata	Besitzer, Version und weitere Schlüssel für eigene Werkzeuge
allowed-tools	vorab erlaubte Werkzeuge; experimentell
Claude-Code-Zusätze	when_to_use, paths (nur bei passenden Dateien), disable-model-invocation (nur manuell), context: fork (eigener Subagent), hooks

```

datenbank-migration/
+-- SKILL.md      Pflicht: Kopf + Anweisungen
+-- references/   Nachschlagewissen,
|   +-- sperren.md je Datei ein Thema
|   +-- rollback.md
+-- scripts/      ausführbare Pruefungen
|   +-- pruefen.py
|   +-- test_pruefen.py
+-- assets/       Vorlagen, Konfiguration
+-- evals/
|   +-- evals.json Testaufgaben
+-- README.md     fuer Menschen
+-- LICENSE       Nutzungsrechte

```

Listing 1. Verzeichnisstruktur eines Skills

Der Agent lädt SKILL.md und bei Bedarf weitere Dateien. README.md richtet sich vor allem an Menschen; wenn der Agent sie liest, beansprucht sie ebenfalls Kontext. Im offenen Standard stimmen Ordnername und name überein. Eine gemeinsame Quelle mit werkzeugspezifischen Verknüpfungen kann doppelte Pflege vermeiden; vor dem Einsatz muss geprüft werden, ob das jeweilige Werkzeug diese Pfade und Verknüpfungen unterstützt.

C. Kopfdaten und Beschreibung

Tabelle X fasst die Felder zusammen. Pflicht sind nur name und description; alle anderen sind optional [26].

Die Beschreibung ist ein wichtiges Auswahlkriterium; explizite Aufrufe und Werkzeugregeln beeinflussen das Laden ebenfalls. Die Formel **Was** + **Wann** + **Auslöser** + **Abgrenzung** ist eine praktische Entwurfsheuristik, angelehnt an die Herstellerempfehlung [27]:

- **Schwach:** „Hilft bei Datenbanken.“
- **Stark:** „Plant und prüft Schemamigrationen der PostgreSQL-Datenbank des Bestellsystems. Verwenden bei neuen Tabellen, Spaltenänderungen oder Indizes und wenn der Nutzer ‚Migration‘, ‚Schema‘ oder ‚DB-Update‘ sagt. Nicht für reine Leseabfragen.“

Die Auswahlhilfe gehört an den Anfang. Claude Code begrenzt die gelistete Kombination aus description und when_to_use standardmäßig auf 1.536 Zeichen und das Beschreibungsbudget auf etwa 1 % des Kontextfensters; beide Werte sind konfigurierbar. Bei Überschreitung entfallen einzelne Beschreibungen, während die Namen erhalten bleiben

[28]. Diese Details gelten für die am 4. Oktober 2026 geprüfte Dokumentation, nicht für den offenen Standard.

D. Aufbau der Anweisungen

Listing 2 zeigt eine Vorlage. In Claude Code bleiben bei Kontextverdichtung höchstens die ersten 5.000 Token je aufgerufenem Skill erhalten; gemeinsam gilt ein Budget von 25.000 Token, sodass ältere Skills ganz entfallen können [28]. Kritische Regeln gehören deshalb nach vorn, Details in Referenzen.

Für den Schreibstil gelten sieben Regeln:

- 1) **Imperativ und konkret:** „Führe pruefen.py aus“ statt „man könnte prüfen“.
- 2) **Regeln knapp begründen:** Eine kurze Begründung kann das Ziel einer Regel verdeutlichen. Ein genereller Übertragungsvorteil gegenüber Großbuchstaben-MUSTs ist in den verwendeten Quellen nicht gemessen; dies ist eine Entwurfsheuristik des Autors.
- 3) **Freiheitsgrad passend wählen:** Bei riskanten Abläufen (Migrationen, Releases) exakte Befehle vorgeben; bei Entwurfsaufgaben Heuristiken und Ziele [27].
- 4) **Eine Bezeichnung je Begriff:** nicht abwechselnd „Feld“, „Spalte“, „Attribut“.
- 5) **Nichts Zeitabhängiges:** statt „ab Juli neue API“ einen Abschnitt „Alte Muster“.
- 6) **Pfade mit Schrägstrich** (scripts/pruefen.py), damit sie auf allen Systemen funktionieren.
- 7) **Nur ergänzen, was das Modell nicht weiß:** Projekt- und Domänenwissen statt Allgemeinwissen, weil zusätzlicher Kontext Kosten verursacht und den Erfolg senken kann [15].

a) *Schlecht gegen gut:* Tabelle XI stellt für dieselbe Aufgabe einen schwachen und einen starken Entwurf gegenüber. Die Gegenüberstellung ist ein illustratives Beispiel des Autors, keine Messung.

E. Skripte und Referenzen

Prüfskripte können eine verlässliche Rückkopplung ermöglichen; dass sie generell der wirksamste Teil eines Skills sind, ist nicht direkt belegt (Abschnitt IV.E). Bei reiner Ausführung muss der Skriptcode nicht in den Modellkontext; beim Review

```

---
name: datenbank-migration
description: >-
  Plant und prueft PostgreSQL-Migrationen des Bestellsystems.
  Verwenden bei Tabellen, Spalten, Indizes und DB-Updates.
  Nicht fuer reine Leseabfragen.
license: Proprietaer, nur intern
metadata:
  owner: team-plattform
  version: "1.2.0"
---

# Datenbank-Migration
Ziel: Migrations- und Wiederherstellungsplan mit geprüften Risiken.

## Vorrang          <- Projektregeln (AGENTS.md) gewinnen bei Widerspruch
## Ablauf           <- nummerierte Checkliste, die der Agent abhakt
1. Bestehende Migrationen und Schema lesen
2. Migration mit Vorwaerts- und Rueckwaertsschritt schreiben
3. Strukturpruefer im Skill-Verzeichnis ausfuehren; Funde beheben
4. Hoechstens drei Pruefrunden; SQL und Rollback separat testen
## Regeln           <- jede Regel mit Begrueendung ("weil ...")
- Spalten nie direkt loeschen, weil laufende Versionen sie noch lesen: erst entkoppeln, dann entfernen.
## Gotchas          <- beobachtete Wiederholungsfehler, je Zeile Fehler -> Korrektur
## Ausgabeformat    <- was der Bericht enthaelt, wie lang er ist
## Referenzen       <- je Datei: wann lesen
- references/sperrern.md: bei grossen Tabellen mit laufenden Zugriffen

```

Listing 2. Vorlage für eine SKILL.md (Kommentare mit Pfeil sind Erläuterungen, nicht Teil der Datei)

Tabelle XI
Gegenüberstellung eines schwachen und eines starken Skills

Element	Schwach	Stark
Beschreibung	„Hilft bei Datenbanken.“	Was, Wann, Auslöser, Abgrenzung (siehe oben)
Ablauf	„Achte auf gute Migrationen und teste sie.“	Nummerierte Schritte mit konkretem Prüfbefehl, Rundenlimit und Bericht offener Funde
Regeln	„IMMER Rollback vorsehen! NIE Daten verlieren!“	„Spalten nie direkt löschen, weil laufende Versionen sie noch lesen: erst entkoppeln, dann entfernen.“
Wissen	Erklärt, was SQL und ein Index sind	Nur Projektspezifisches: Namensschema, Sperrverhalten bei großen Tabellen
Prüfung	keine	Skript mit festen Exit-Codes und begrenzter Ausgabe
Länge	900 Zeilen Wiki-Kopie	unter 100 Zeilen, Details in Referenzen

oder Debugging wird er oft dennoch gelesen. Daraus folgen praktische Empfehlungen:

- Im Skill eindeutig sagen, ob ein Skript **ausgeführt** oder **als Referenz gelesen** werden soll.
- Standardmäßig nur lesend; schreibende Aktionen nur mit ausdrücklicher Option.
- Kompakte, begrenzte Ausgabe (--limit, Filter, optional JSON), Fundstellen als `datei:zeile`; das entspricht dem Prinzip kompakter Schnittstellen bei [11].
- Eindeutige Exit-Codes (0 = sauber, 1 = Funde, 2 = Bedienfehler) und hilfreiche Fehlermeldungen statt Abstürzen.
- Möglichst nur Standardbibliothek; sonst Abhängigkeiten im Feld `compatibility` nennen.
- Keine unbegründeten Schwellwerte; jede Konstante mit kurzer Begründung.
- Kein Netzwerkzugriff und keine Zugangsdaten ohne zwingenden Grund, weil Skripte das Hauptrisiko darstellen [19].
- Selbst testen: absichtlich fehlerhafte Beispiele müssen gefunden werden; echter Code muss stichprobenartig

auf Fehlalarme geprüft werden, denn ein Prüfskript ist selbst Code.

Referenzdateien behandeln je ein Thema, sind nur eine Ebene tief von SKILL.md verlinkt (bei tieferer Verschachtelung lesen Agenten Dateien oft nur teilweise) und erhalten ab etwa 100 Zeilen ein Inhaltsverzeichnis [27].

F. Testen und Beobachten

Ein Skill ist erst dann belegt wirksam, wenn zwei Dinge getrennt gemessen wurden: ob er *ausgelöst* wird und ob er *hilft*, wenn er geladen ist. Als Vorbild für das Messdesign dient SkillsBench: dieselbe Aufgabe ohne Skill und mit Skill, mehrere Läufe, mehrere Modelle [16], [18].

Auslösetests (Trigger-Tests). Ein Testsatz enthält *positive* Anfragen (Skill soll laden), *negative* Anfragen (Skill soll nicht laden, etwa Leseabfragen) und *Grenzfälle* (Formulierungen ohne die Schlüsselwörter der Beschreibung). Jede Anfrage wird mehrfach ausgeführt, weil das Modell nicht deterministisch entscheidet. Gemessen wird die Aktivierungsrate je Gruppe; als Aktivierung zählt nur ein erfolgreicher Skill-Aufruf oder eine vollständige Lektüre der SKILL.md, nicht

Tabelle XII
Anti-Muster beim Einsatz von Skills

Anti-Muster	Folge	Besser
Skill als Ablage für alles (Wiki-Kopie, komplette API-Doku)	verwässerte Anweisungen, hohe Kosten	Wenige passende Skills, kompakte Inhalte und Referenzen [16]
Vage Beschreibung („Hilft bei Code“)	Skill wird nicht geladen	Was + Wann + Auslöser + Abgrenzung
Viele ähnliche Skills	falsche Auswahl, gekürzte Beschreibungen [17]	zusammenlegen, klare Grenzen
Pflichtprüfung nur im Skill	wird in einem Teil der Fälle vergessen [31]	zusätzlich Hook oder CI
Lange Regeldatei statt Skill	kein genereller Erfolgsgewinn, über 20 % Mehrkosten [15]	Regeldatei kurz, Details in Skills
Erklären, was das Modell ohnehin weiß	Token und Ablenkung	nur Projekt- und Domänenwissen
Vom Modell erzeugter Skill ohne Prüfung	in den untersuchten Konfigurationen kein Vorteil [16]	aus echten Fehlern kuratieren
Starre MUST-Ketten ohne Begründung	Ziel einer Regel bleibt unklar (Entwurfsheuristik)	Regel knapp begründen
Tief verschachtelte Referenzen	unvollständiges Lesen (Herstellerhinweis)	eine Ebene, Inhaltsverzeichnis
Ungeprüfte Skills aus Marktplätzen	26,1 % vom Detektor als verwundbar klassifiziert [19]	interne Registry mit Review
Zugangsdaten oder Kundendaten im Skill	Datenabfluss	Umgebungsvariablen, Secret-Store
Skripte mit ungefilterter Ausgabe	voller Kontext, höhere Kosten	Begrenzung, Zusammenfassung
Skill ohne Testaufgaben	Wirkung unbekannt; 13 von 87 Aufgaben wurden mit Skill schlechter [16]	Evals mit Basislinie, mehrere Modelle
Widersprüchliche Skills ohne Vorrang	unvorhersehbares Verhalten (Schlussfolgerung des Autors)	Vorrang festhalten, Überschneidung vermeiden

die bloße Nennung des Namens. Ein Vorlagenformat steht in Abschnitt X.

Wirksamkeitstests. Mindestens drei realistische Aufgaben mit vorab festgelegten, möglichst per Skript prüfbaren Kriterien, jeweils mit und ohne Skill, mehrere Läufe, alle eingesetzten Modelle; neben Qualität auch Token und Laufzeit. Kriterien nicht erst nach dem Lauf festlegen.

Beobachtbarkeit. Werkzeugprotokolle und passende Hooks können erfolgreiche Skill-Aufrufe bzw. Dateizugriffe dokumentieren [29]. Ein Kennsatz im Abschlussbericht ist nur ein Hinweis und kein Nachweis des Ladens.

Fehlersuche, wenn ein Skill nicht greift: (1) Ist er sichtbar (richtiges Verzeichnis, gültiger Kopf, Name gleich Ordner)? (2) Wurde die Beschreibung gekürzt (Abschnitt VI.C)? (3) Enthält die Beschreibung die Wörter, die Nutzer tatsächlich schreiben? (4) Konkurriert ein ähnlicher Skill? (5) Hilft ein Verweis im Immer-Kontext oder ein Hook?

G. Richtig nutzen im Alltag

Für Entwickler:innen, die Skills anwenden, gelten diese Regeln:

- 1) **Bei klarer Aufgabe den Skill direkt aufrufen** (/name oder Nennung im Prompt), statt auf die automatische Auslösung zu hoffen, weil diese unzuverlässig ist [31].

- 2) **Eine Aufgabe je Sitzung.** Fremde Themen in einer neuen Sitzung beginnen, damit alter Kontext nicht mitbezahlt wird und nicht stört [25].
- 3) **Ergebnisse an Prüfungen messen,** nicht an der Überzeugungskraft der Antwort; wahrgenommene und tatsächliche Wirkung können weit auseinanderfallen [4].
- 4) **Wiederholte Fehler melden.** Jeder zweite gleiche Fehler wird eine Gotcha-Zeile.
- 5) **Kosten über das Modell steuern.** Kleinere Modelle mit gutem Skill können stärkere Basislinien ohne Skill erreichen [16]; für Routine reicht oft das günstigere Modell.
- 6) **Werkzeugfunde beurteilen.** Fehlalarme dürfen mit Begründung bestehen bleiben.

H. Anti-Muster

Tabelle XII listet, wie Skills *nicht* eingesetzt werden sollten, jeweils mit Folge und Alternative.

I. Lebenszyklus und Freigabe im Unternehmen

Skills sind Code und brauchen denselben Lebenszyklus:

- 1) **Bedarf belegen:** wiederkehrende Fehler ohne Skill sammeln.
- 2) **Testaufgaben zuerst:** mindestens drei realistische Aufgaben mit prüfbaren Kriterien.

- 3) **Minimaler Entwurf:** nur so viel, dass die Testaufgaben bestehen.
- 4) **Messen:** mit und ohne Skill, mehrere Läufe, alle eingesetzten Modelle; neben Qualität auch Token und Laufzeit. Da einzelne Aufgaben mit Skill schlechter werden können [16], ist die Messung Pflicht und kein Luxus.
- 5) **Review und Freigabe** nach der Checkliste unten.
- 6) **Ausrollen** über eine interne Registry oder das Repository.
- 7) **Beobachten:** Wird der Skill ausgelöst? Welche Fehler wiederholen sich?
- 8) **Pflegen:** Gotchas ergänzen, Version erhöhen, Änderungen gegen die Testaufgaben prüfen.
- 9) **Stillegen**, wenn der Ablauf entfällt oder der Skill nicht mehr messbar hilft.

Freigabe-Checkliste:

- Name gleich Ordnername; Beschreibung mit Was, Wann, Auslösern und Abgrenzung.
- SKILL.md unter 500 Zeilen, Wichtigstes oben, keine zeitabhängigen Aussagen.
- Referenzen eine Ebene tief, je ein Thema, Inhaltsverzeichnis ab 100 Zeilen.
- Skripte gelesen, getestet, standardmäßig nur lesend, ohne Zugangsdaten und ohne Netzwerkzugriff.
- Alle Dateien (auch Referenzen) auf Prompt Injection geprüft.
- Mindestens drei Testaufgaben mit Vergleich gegen eine Basislinie; Trigger-Tests mit positiven und negativen Anfragen.
- Besitzer, Version und Lizenz bzw. Vertraulichkeit eingetragen.
- Pflichtprüfungen zusätzlich als Hook oder in der CI.
- Keine inhaltliche Überschneidung oder Widersprüche mit bestehenden Skills.

VII. GRENZEN UND OFFENE FRAGEN

Grenzen dieser Arbeit.

- **Keine eigenen Messungen:** Alle Zahlen stammen aus den zitierten Quellen und gelten für deren Modelle, Werkzeuge und Aufgaben.
- **Junges Feld:** Die vier Studien, die Skills direkt untersuchen [16], [17], [18], [19], sind Preprints; Befunde können sich bei Begutachtung oder mit neuen Modellen ändern.
- **Narrative Auswahl:** Ein vollständiges Such- und Screeningprotokoll fehlt; Quellen können übersehen worden sein.
- **Herstellerbezug:** Werkzeugdetails (Limits, Zusatzfelder, Vorrang) stammen aus Herstellerdokumentation und ändern sich schnell.
- **Übertragungen:** Die Wirkung von Rückkopplung und Kontextlänge ist allgemein belegt, nicht speziell für Skills. Sicherheitsmaßnahmen und Konfliktregeln sind Ableitungen, keine gemessenen Befunde.

Offene Forschungsfragen.

- 1) Wie hoch ist die Aktivierungsrate von Skills über Modelle, Werkzeuge und Beschreibungsstile hinweg? Vercel, SkillsBench und Jiang et al. verwenden unterschiedliche Aufgaben und Kennzahlen.
- 2) Wie lässt sich die Aktivierung ohne zusätzliche Kontextkosten verbessern?
- 3) Wie bleiben Skills mit dem Code, den sie beschreiben, automatisch aktuell?
- 4) Wie entwickelt sich der Nutzen von Skills in großen, proprietären Codebasen über längere Zeiträume? Der Zugewinn in der Softwareentwicklung war in SkillsBench vergleichsweise klein [16]; ob projektspezifische Skills hier mehr bringen, ist ungeklärt.
- 5) Wie wirken Skills mit Konflikten untereinander, und welche Vorrangmechanismen helfen?

VIII. SCHLUSSFOLGERUNG

Agent Skills sind ein wirksames, werkzeugübergreifendes Mittel, um KI-Agenten Wissen zugänglich zu machen, aber nur unter Bedingungen. Bei modellgesteuerter Aktivierung entscheidet der Agent anhand verfügbarer Beschreibungen, ob er einen Skill öffnet (RQ1). Skills helfen, wenn sie von Menschen kuratiert, fokussiert und als Arbeitsablauf statt als Lexikon geschrieben sind; selbstgenerierte Skills unterliegen in den untersuchten Konfigurationen der Basislinie, und in der Softwareentwicklung liegt der Zugewinn unter dem Gesamtdurchschnitt (RQ2). Aktivierung und korrekte Anwendung hängen vom Setup ab; kurzer relevanter Kontext und überprüfbare Abläufe sind deshalb sinnvolle Gestaltungsziele (RQ3). Im Unternehmen müssen sie wie Code behandelt werden: versioniert, getestet, gemessen und als Lieferkettenrisiko abgesichert (RQ4).

Für den Einstieg in einem Team genügen fünf Schritte: wiederkehrende Fehler sammeln, drei Testaufgaben formulieren, einen kleinen Skill nach Listing 2 schreiben, ihn mit und ohne Skill messen und Pflichtprüfungen zusätzlich als Hook absichern.

IX. ANHANG A: VOLLSTÄNDIGER BEISPIEL-SKILL

Der folgende Skill illustriert den Leitfaden. Er besteht aus der SKILL.md (Listing 3) und einem lesenden Strukturprüfer (Listing 4); projektspezifische Schema-, Sperr- und Rollback-Informationen müssen ergänzt werden. Seine Wirksamkeit wurde nicht empirisch evaluiert. PostgreSQL erlaubt `CREATE INDEX CONCURRENTLY` nicht innerhalb eines Transaktionsblocks [32]. Eine neue Pflichtspalte braucht eine passende Einführungsstrategie; ein Default ist eine Möglichkeit, ein schrittweises Backfill eine andere [33]. Das Beispiel garantiert weder Datenverlustfreiheit noch einen sicheren Rollback.

```
---
name: datenbank-migration
description: >-
  Plant und prueft PostgreSQL-Migrationen des Bestellsystems.
  Verwenden bei Tabellen, Spalten, Indizes und DB-Updates.
  Nicht fuer reine Leseabfragen.
compatibility: Erfordert Python 3.10+ und eine PostgreSQL-Testdatenbank.
license: Proprietaer, nur intern
metadata:
  owner: team-plattform
  version: "1.0.0"
---

# Datenbank-Migration
Ziel: Migrations- und Wiederherstellungsplan mit geprüften Risiken.

## Vorrang
Regeln in AGENTS.md gewinnen bei Widerspruch zu diesem Skill.

## Ablauf
1. Lies das aktuelle Schema und die letzten drei Migrationen
   im Verzeichnis db/migrations/.
2. Schreibe die neue Migration mit Abschnitt `-- up` und `-- down`.
3. Fuehre den Strukturpruefer aus. Er liegt im Skill-Verzeichnis;
   verwende dessen absoluten Pfad, unabhaengig vom Arbeitsverzeichnis:
   `python <skill-verzeichnis>/scripts/pruefen.py db/migrations/<datei>.sql`
4. Korrigiere Funde und pruefe erneut, hoechstens drei Runden.
   Exit-Code 0 bestaetigt nur die geprüfte Dateistruktur.
5. Pruefe up und down in einer isolierten Testdatenbank; bewerte
   Sperrren, Altversionen und Datenverlust. Aendere keine Produktion.
6. Berichte in zehn Zeilen: Aenderung, Risiko, Tests, offene Funde.

## Regeln
- Spalten erst entkoppeln, dann spaeter entfernen; pruefe vorher,
  ob Altversionen die Spalte noch verwenden und Daten gesichert sind.
- Pflichtspalten mit Default oder gestuftem Backfill einfuehren;
  teste vorhandene Daten und Schreibzugriffe alter Versionen.
- Fuer Indizes auf aktiven grossen Tabellen CONCURRENTLY pruefen;
  dies erlaubt Schreibzugriffe, aber keinen Transaktionsblock.

## Gotchas
- down fehlt oder ist leer: Rueckwaertsschritt schreiben und testen.
- CONCURRENTLY im Transaktionsblock: separate Migration verwenden.
- Datenumformung ohne Wiederherstellung: Sicherungsplan klaeren.

## Ausgabeformat
Maximal zehn Zeilen: Aenderung, Risiko, Rollback, Tests, offene Funde.

## Referenzen
- Lies die Projektdokumentation zum Schema und Rollout-Verfahren.
- Pruefe PostgreSQL-Dokumentation zu CREATE INDEX und ALTER TABLE.
```

Listing 3. Beispiel-Skill datenbank-migration (SKILL.md)

```
#!/usr/bin/env python3
"""Prueft nur up/down-Abschnitte, nicht SQL-Sicherheit oder Rollback.
Exit-Codes: 0 = sauber, 1 = Funde, 2 = Bedienfehler."""
import re
import sys

LIMIT = 20 # Ausgabe begrenzen; kein SQL-Parser

def pruefe(pfad):
    with open(pfad, encoding="utf-8") as f:
        zeilen = f.read().splitlines()
        marker = [(i, m[1].lower()) for i, z in enumerate(zeilen)
                    if (m := re.fullmatch(r"\s*--\s*(up|down)\s*", z, re.I))]
        if [name for _, name in marker] != ["up", "down"]:
            return [(1, "Erwartet: genau '-- up', dann '-- down'")]
        funde = []
        for j, (start, name) in enumerate(marker):
            ende = marker[j + 1][0] if j == 0 else len(zeilen)
            inhalt = "\n".join(zeilen[start + 1:ende])
            # Heuristik: Kommentare entfernen; SQL-Literale werden nicht geparkt.
            inhalt = re.sub(r"/\s*\s*/|--[^\n]*", "", inhalt, flags=re.S)
            if not inhalt.strip("\t\r\n;"):
                funde.append((start + 1, f"Abschnitt '{name}' ist leer"))
        return funde

def main():
    if len(sys.argv) != 2:
        print("Aufruf: pruefen.py <datei.sql>", file=sys.stderr)
        return 2
    try:
        funde = pruefe(sys.argv[1])
    except (OSError, UnicodeError) as e:
        print(f"Datei nicht lesbar: {e}", file=sys.stderr)
        return 2
    for nr, msg in funde[:LIMIT]:
        print(f"{sys.argv[1]}:{nr}: {msg}")
    if len(funde) > LIMIT:
        print(f"{len(funde) - LIMIT} weitere Funde nicht angezeigt")
    return 1 if funde else 0

if __name__ == "__main__":
    sys.exit(main())
```

Listing 4. Illustrativer Strukturprüfer; Exit-Code 0 ist kein SQL-Sicherheitsnachweis

X. ANHANG B: VORLAGE FÜR AUSLÖSETESTS

Vorlage für evals/evals.json mit positiven, negativen und Grenzfall-Anfragen. Jede Anfrage wird mehrfach ausgeführt; gezählt wird nur ein erfolgreicher Skill-Aufruf oder eine vollständige Lektüre der SKILL.md.

```
{
  "skill": "datenbank-migration",
  "runs_per_prompt": 5,
  "cases": [
    {"prompt": "Fuege der Tabelle orders eine Spalte status hinzu",
     "expect_load": true, "kind": "positiv"},
    {"prompt": "Schreibe ein DB-Update fuer den neuen Index",
     "expect_load": true, "kind": "positiv"},
    {"prompt": "Wie viele Bestellungen gab es gestern?",
     "expect_load": false, "kind": "negativ"},
    {"prompt": "Erklaere den Unterschied zwischen INNER und LEFT JOIN",
     "expect_load": false, "kind": "negativ"},
    {"prompt": "Die Kundentabelle braucht ein zusaetzliches Feld",
     "expect_load": true, "kind": "grenzfall"}
  ]
}
```

Listing 5. Vorlage für einen Trigger-Testsatz

XI. ANHANG C: GLOSSAR

Tabelle XIII
Glossar der verwendeten Begriffe

Begriff	Bedeutung in dieser Arbeit
Agent Skill	Verzeichnis mit <code>SKILL.md</code> , optionalen Referenzen, Skripten und Assets, das ein Agent bei Bedarf lädt
Immer-Kontext	Für die Aufgabe dauerhaft bereitgestellte Regeln; der Geltungsbereich hängt vom Werkzeug ab
Progressive Offenlegung	dreistufiges Laden: Metadaten, Anweisungen, Ressourcen
Aktivierung	Der Agent lädt den Skill tatsächlich für eine Aufgabe
Prüfskript	Skript im Skill, das Qualität misst und Funde meldet
Hook	deterministisch ausgeführte Aktion bei einem Ereignis des Agenten
Gotcha	dokumentierter, wiederkehrender Fehler mit Korrektur
Subagent	Agent mit eigenem Kontext für eine Teilaufgabe
MCP	Protokoll, über das Agenten externe Werkzeuge und Systeme nutzen
Context Rot	Leistungsabfall bei wachsender Eingabelänge [25]
RAG	Abruf relevanter Textstellen per Suche, um sie dem Modell mitzugeben
Evidenzstärke	stark / mittel / schwach, siehe Abschnitt II

LITERATURVERZEICHNIS

- [1] S. Peng, E. Kalliamvakou, P. Cihon, und M. Demire, „The Impact of AI on Developer Productivity: Evidence from GitHub Copilot“, *arXiv preprint arXiv:2302.06590*, 2023, [Online]. Verfügbar unter: <https://arxiv.org/abs/2302.06590>
- [2] E. Paradis, K. Grey, Q. Madison, D. Nam, A. Macvean, und others, „How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial“, in *Proceedings of the IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, [Online]. Verfügbar unter: <https://doi.org/10.1109/ICSE-SEIP66354.2025.00060>
- [3] Z. K. Cui, M. Demire, S. Jaffe, L. Musolff, S. Peng, und T. Salz, „The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers“, [Online]. Verfügbar unter: <https://doi.org/10.2139/ssrn.4945566>
- [4] J. Becker, N. Rush, E. Barnes, und D. Rein, „Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity“, *arXiv preprint arXiv:2507.09089*, 2025, [Online]. Verfügbar unter: <https://arxiv.org/abs/2507.09089>
- [5] DORA, „State of AI-assisted Software Development“, technical report, 2025, [Online]. Verfügbar unter: <https://dora.dev/research/2025/dora-report/>
- [6] W. Harding, „AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones“, technical report, 2025, [Online]. Verfügbar unter: https://www.gitclear.com/ai_assistant_code_quality_2025_research
- [7] Anthropic, „Equipping Agents for the Real World with Agent Skills“, Zugegriffen: 4. Oktober 2026, [Online]. Verfügbar unter: <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- [8] N. F. Liu u. a., „Lost in the Middle: How Language Models Use Long Contexts“, *Transactions of the Association for Computational Linguistics*, Bd. 12, S. 157–173, 2024, [Online]. Verfügbar unter: https://doi.org/10.1162/tacl_a_00638
- [9] A. Madaan, N. Tandon, P. Gupta, und others, „Self-Refine: Iterative Refinement with Self-Feedback“, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, [Online]. Verfügbar unter: <https://arxiv.org/abs/2303.17651>
- [10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, und S. Yao, „Reflexion: Language Agents with Verbal Reinforcement Learning“, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, [Online]. Verfügbar unter: <https://arxiv.org/abs/2303.11366>
- [11] J. Yang u. a., „SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering“, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, [Online]. Verfügbar unter: <https://arxiv.org/abs/2405.1793>
- [12] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, und R. Karri, „Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions“, in *IEEE Symposium on Security and Privacy (SP)*, 2022, S. 754–768, [Online]. Verfügbar unter: <https://arxiv.org/abs/2108.09293>
- [13] G. Wang u. a., „Voyager: An Open-Ended Embodied Agent with Large Language Models“, *Transactions on Machine Learning Research*, 2024, [Online]. Verfügbar unter: <https://openreview.net/forum?id=ehfRiF0R3a>
- [14] C. Bird u. a., „Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools“, *ACM Queue*, 2023, doi: 10.1145/3582083.
- [15] T. Gloaguen, N. Mündler, M. N. Müller, V. Raychev, und M. Vechev, „Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?“, in *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems*, 2026, [Online]. Verfügbar unter: <https://arxiv.org/abs/2602.11988v3>
- [16] X. Li, Y. Liu, W. Chen, B. You, Z. Di, und others, „SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks“, *arXiv preprint arXiv:2602.12670v4*, 2026, [Online]. Verfügbar unter: <https://arxiv.org/abs/2602.12670v4>
- [17] Z. Jiang u. a., „Demystifying Agent Skills: Why They Work – Until They Don’t“, *arXiv preprint arXiv:2608.14036*, 2026, [Online]. Verfügbar unter: <https://arxiv.org/abs/2608.14036>
- [18] M. Shaposhnikov, N. Fortuin, S. Stipich, M. I. Gorinova, A. Heineike, und R. Willoughby, „A Framework for Evaluating Agentic Skills at Scale“, *arXiv preprint arXiv:2606.17819*, 2026, [Online]. Verfügbar unter: <https://arxiv.org/abs/2606.17819>
- [19] Y. Liu u. a., „Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale“, *arXiv preprint arXiv:2601.10338*, 2026, [Online]. Verfügbar unter: <https://arxiv.org/abs/2601.10338>
- [20] D. Jaroslawicz, B. Whiting, P. Shah, und K. Maamari, „How Many Instructions Can LLMs Follow at Once?“, *arXiv preprint arXiv:2507.11538*, 2025, [Online]. Verfügbar unter: <https://arxiv.org/abs/2507.11538>
- [21] T. Gan und Q. Sun, „RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation“, *arXiv preprint arXiv:2505.03275*, 2025, [Online]. Verfügbar unter: <https://arxiv.org/abs/2505.03275>
- [22] S. Blyth, S. A. Licorish, C. Treude, und M. Wagner, „Static Analysis as a Feedback Loop: Enhancing LLM-Generated Code Beyond Correctness“, *arXiv preprint arXiv:2508.14419*, 2025, [Online]. Verfügbar unter: <https://arxiv.org/abs/2508.14419>
- [23] M. Tran u. a., „Characterizing the Quality Profile of AI-Generated C++ in Production“, *arXiv preprint arXiv:2608.06640*, 2026, [Online]. Verfügbar unter: <https://arxiv.org/abs/2608.06640>
- [24] M. Kozak, R. Zilouchian Moghaddam, und S. Sivaraman, „When Developer Aid Becomes Security Debt: A Systematic Analysis of Insecure Behaviors in LLM Coding Agents“, *arXiv preprint arXiv:2507.09329*, 2025, [Online]. Verfügbar unter: <https://arxiv.org/abs/2507.09329>
- [25] K. Hong, A. Troynikov, und J. Huber, „Context Rot: How Increasing Input Tokens Impacts LLM Performance“, technical report, 2025, [Online]. Verfügbar unter: <https://www.trychroma.com/research/context-rot>
- [26] Agent Skills Project, „Agent Skills Specification“, Zugegriffen: 4. Oktober 2026, [Online]. Verfügbar unter: <https://agentskills.io/specification>
- [27] Anthropic, „Skill Authoring Best Practices“, Zugegriffen: 4. Oktober 2026, [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices>
- [28] Anthropic, „Extend Claude with Skills“, Zugegriffen: 4. Oktober 2026, [Online]. Verfügbar unter: <https://code.claude.com/docs/en/skills>
- [29] Anthropic, „Hooks reference“, Zugegriffen: 4. Oktober 2026, [Online]. Verfügbar unter: <https://code.claude.com/docs/en/hooks>
- [30] Anthropic, „How We Built Our Multi-Agent Research System“, Zugegriffen: 4. Oktober 2026, [Online]. Verfügbar unter: <https://www.anthropic.com/engineering/multi-agent-research-system>
- [31] Vercel, „AGENTS.md Outperforms Skills in Our Agent Evals“, Zugegriffen: 4. Oktober 2026, [Online]. Verfügbar unter: <https://vercel.com/blog/agents-md-outperforms-skills-in-our-agent-evals>
- [32] PostgreSQL Global Development Group, „PostgreSQL 18 Documentation: CREATE INDEX“, [Online]. Verfügbar unter: <https://www.postgresql.org/docs/18/sql-createindex.html>
- [33] PostgreSQL Global Development Group, „PostgreSQL 18 Documentation: Modifying Tables“, [Online]. Verfügbar unter: <https://www.postgresql.org/docs/18/ddl-alter.html>